

MIDI Maze Serial Protocol

A Reverse-Engineering Reference

MIDI Maze Serial Protocol

A Reverse-Engineering Reference

This document is a reverse-engineering notebook describing how **MIDI Maze** initializes and conducts

multiplayer gameplay over the Atari SIO MIDI serial ring.

All behavior described here is derived directly from the **MIDI Maze ROM disassembly**.

Addresses are given in `Label (Lxxxx)` form where available.

1. Scope and Assumptions

- * Transport: Atari SIO raw POKEY serial at MIDI rate (31250 baud)
- * Network model: Unidirectional ring; every byte is forwarded
- * No framing, no checksums during gameplay
- * Byte order is semantic

2. Ring Network Model

Each machine:

1. Injects its own bytes
2. Forwards all received bytes unchanged
3. Maintains per-player state indexed by ring position

A single lost, duplicated, or reordered byte corrupts the stream globally.

3. Game Setup Protocol (Summary)

See detailed setup phases in earlier sections:

- * Master election
- * Configuration broadcast
- * Bot setup (`\$84`)
- * Transition to gameplay (`\$87`)

Evidence: Setup dispatch logic in `SetupDispatcher (L9B3C)`.

4. Transition to Gameplay

CMD_START_GAME (\$87) switches all machines into gameplay mode.

Before live play:

- * A 7-byte parameter block is exchanged per player
- * Ensures identical tuning values

Evidence: Gameplay entry relay loop in `GameplayInitRelay (L9772–L9808)`.

Gameplay Entry Parameter Block

Byte	Name	Meaning
------	------	---------

----	-----	-----
------	-------	-------

0	FireCooldown	Weapon fire cooldown
---	--------------	----------------------

1	ReloadTimer	Weapon reload timing
---	-------------	----------------------

2	ProjectileLife	Projectile lifetime
---	----------------	---------------------

3	WeaponMode	Weapon mode selector
---	------------	----------------------

4	MoveSpeedFlag	Movement speed modifier
---	---------------	-------------------------

5	ProjectileSpeedFlag	Projectile speed modifier
---	---------------------	---------------------------

6	TurnRate	Rotation step size
---	----------	--------------------

5. Steady-State Gameplay Communication

Gameplay is a **continuous byte stream**.

Update Types

* **Normal update**: 1 byte

* **Extended update**: 2 bytes

Evidence: Send/receive loop in `GameplayCommLoop (L8129–L81C0)`.

6. Controller Byte Format

Bit	Meaning
-----	---------

----	-----
------	-------

0	Joystick Up
---	-------------

1	Joystick Down
---	---------------

2	Joystick Left
---	---------------

3	Joystick Right
---	----------------

4	Fire
---	------

5–6	Unused
-----	--------

| 7 | Extended payload follows |

Evidence: Controller builder in `BuildControllerByte (L8129)`.

7. Extended Payload Byte

Only valid if controller bit7 = 1.

| Range | Meaning |

|-----|-----|

| \$00–\$7F | Text / control |

| \$80–\$FE | Command |

| \$FF | No payload |

Evidence: Payload latch logic in `ReceiveExtendedPayload (L81F0)`.

8. In-Game Payload Commands

Latched into `PendingCommand (L3EE7)` and dispatched by:

****CommandDispatcher (L9598)****

| Cmd | Name | Description |

|----:|-----|-----|

| \$81 | ClearState | Clears transient gameplay arrays |

| \$82 | HoldSync | Displays “PLEASE HOLD” |

| \$84 | Resync | Ring-wide resynchronization |

| \$86 | RosterExchange | Structured roster/text exchange |

9. Resynchronization (\$84) Deep Dive

Trigger

* Any machine may inject `84` as payload

Behavior

* Master (`PlayerIndex == 0`) transmits

* Slaves receive only

Wire Flow

1. `84` circulates

2. Master enters transmit path

3. `83` marker sent

4. Configuration payload follows

5. Slaves wait on `83`

Evidence:

* Master branch: `ResyncTransmit (L9684)`

* Slave branch: `ResyncReceive (L96D9)`

10. Timeouts and Errors

Timeout occurs when expected byte does not arrive before deadline.

* Deadline = current tick + 15 VBLANKs (~250ms NTSC)

* Sets `ErrorCode (L3ED2) = \$C7`

Evidence: Timeout check in `GameplayReceiveWait (L81A0)`.

11. Error / Status Codes

| Code | Message | Phase |

|-----|-----|-----|

| \$02 | Game terminated | Both |

| \$03 | Maze too small | Setup |

| \$04 | Network error | Setup |

| \$05 | Too many machines | Setup |

| \$06 | No drones allowed | Setup |

| \$07 | Checksum error | Setup |

| \$08 | Device not responding | Setup |

| \$09 | No such device | Setup |

| \$0A | Can't sync | Setup |

| \$C7 | I/O timeout | Gameplay |

Printed via `PrintStatusMessage (L92B3)`.

12. Game Termination

No explicit "game over" packet exists.

Termination occurs when:

- * An error is raised
- * A resync fails
- * Communication stalls

Control returns to menu via `ResetToMenu (L8050)`.

Appendix A: Key Routines

- * `CommandDispatcher (L9598)`
- * `GameplayCommLoop (L8129)`
- * `ResyncTransmit (L9684)`
- * `ResyncReceive (L96D9)`
- * `PrintStatusMessage (L92B3)`
- * `ResetToMenu (L8050)`

Appendix B: Fragility Notes

- * Stream is position-dependent
- * Extended payload misalignment corrupts ring
- * Recovery assumes ring responsiveness